

Assignment 2

We start by creating a small database with ai-generated mock data from scratch and then transforming that initial unnormalized dataset through the three normalization stages in order to eliminate data redundancy, prevent update anomalies, and enforce relational integrity.

```
8
9  CREATE TABLE Flat_AI_Worker_Data (
10     worker_id INT,
11     worker_salary DECIMAL(10,2),
12     project_id INT,
13     project_name VARCHAR(100),
14     company_id INT,
15     company_name VARCHAR(100),
16     industry_id INT,
17     industry_name VARCHAR(100),
18     ai_technologies_used VARCHAR(255)
19 );
20
21
22 INSERT INTO Flat_AI_Worker_Data VALUES
23 (1, 75000.00, 501, 'AI Migration', 101, 'TechCorp', 10, 'Software & Data', 'ChatGPT, TensorFlow, Claude'),
24 (1, 75000.00, 502, 'Data Pipeline', 101, 'TechCorp', 10, 'Software & Data', 'ChatGPT, TensorFlow'),
25 (2, 90000.00, 501, 'AI Migration', 102, 'FinDataSolutions', 10, 'Software & Data', 'PyTorch, OpenAI API');
```

First Normal Form (1NF)

We eliminated multi-valued attributes to ensure every cell contains a single, atomic value. In our unnormalized setup, the ai_technologies_used column stored a comma-separated list of tools for a single company row. We changed this by putting those values out into a dedicated bridge table, which ensures that each row represents a single, indivisible relationship.

```
1  CREATE TABLE Company_AI_Technology_1NF (
2     company_id INT,
3     technology_name VARCHAR(100),
4     PRIMARY KEY (company_id, technology_name)
5 );
6
7  INSERT INTO Company_AI_Technology_1NF (company_id, technology_name) VALUES
8  (101, 'ChatGPT'),
9  (101, 'TensorFlow'),
10 (101, 'Claude'),
11 (102, 'PyTorch'),
12 (102, 'OpenAI API');
```

Second Normal Form (2NF)

We removed partial dependencies to ensure all non-key attributes depend on the entire primary key. In a composite key scenario where rows are identified by both worker_id and project_id, attributes like worker_salary depend solely on worker_id rather than the whole key combination. We split that into two distinct tables to make sure the composite relationship still exists.

```
1  CREATE TABLE Worker_2NF (
2     worker_id INT PRIMARY KEY,
3     worker_salary DECIMAL(10,2),
4     company_id INT
5 );
6
7  INSERT INTO Worker_2NF (worker_id, worker_salary, company_id)
8  SELECT DISTINCT worker_id, worker_salary, company_id
9  FROM Flat_AI_Worker_Data;
10
11
12 CREATE TABLE Project_Assignment_2NF (
13     worker_id INT,
14     project_id INT,
15     project_name VARCHAR(100),
16     PRIMARY KEY (worker_id, project_id)
17 );
18
19 INSERT INTO Project_Assignment_2NF (worker_id, project_id, project_name)
20 SELECT DISTINCT worker_id, project_id, project_name
21 FROM Flat_AI_Worker_Data;
```

Third Normal Form (3NF)

We eliminated transitive dependencies where non-key attributes depend on other non-key attributes instead of on the primary key. In our initial design, fields like industry_name depended on industry_id inside the company record. We fixed this by extracting industry details into their own table and retaining only the foreign key reference in the company entity.

```
1  CREATE TABLE Industry_3NF (
2     industry_id INT PRIMARY KEY,
3     industry_name VARCHAR(100)
4 );
5
6  INSERT INTO Industry_3NF (industry_id, industry_name)
7  SELECT DISTINCT industry_id, industry_name
8  FROM Flat_AI_Worker_Data;
9
10 CREATE TABLE Company_3NF (
11     company_id INT PRIMARY KEY,
12     company_name VARCHAR(100),
13     industry_id INT,
14     FOREIGN KEY (industry_id) REFERENCES Industry_3NF(industry_id)
15 );
16
17 INSERT INTO Company_3NF (company_id, company_name, industry_id)
18 SELECT DISTINCT company_id, company_name, industry_id
19 FROM Flat_AI_Worker_Data;
```

